

**UNIwersYTET RZESZOWSKI**  
**WYDZIAŁ NAUK ŚCISŁYCH I TECHNICZNYCH**  
**INSTYTUT INFORMATYKI**



*Kacper Bułaś, Mateusz Bocak*  
127754, 125104

*Informatyka*

*Klasyfikacja gestów języka migowego przy użyciu Sign Language  
MNIST*

Grupa: 1

Prowadzący  
dr Zbigniew Gomółka

Rzeszów 2025



## Spis treści

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1. Cel projektu i wprowadzenie teoretyczne.....</b>    | <b>7</b>  |
| 1.1. Cel projektu .....                                   | 7         |
| 1.2. Wprowadzenie teoretyczne.....                        | 7         |
| 1.2.1. Zbiór danych Sign Language MNIST.....              | 7         |
| 1.2.2. Metody klasyfikacji.....                           | 8         |
| 1.2.3. Detekcja dłoni — MediaPipe.....                    | 8         |
| 1.2.4. Normalizacja cech.....                             | 8         |
| <b>2. Opis zastosowanej metody .....</b>                  | <b>9</b>  |
| 2.1. Ogólny schemat systemu .....                         | 9         |
| 2.2. Potok przetwarzania danych .....                     | 9         |
| 2.3. Warunki pomiarowe przy inferowaniu z kamery .....    | 10        |
| <b>3. Implementacja systemu .....</b>                     | <b>11</b> |
| 3.1. Struktura projektu .....                             | 11        |
| 3.2. Interfejs wiersza poleceń .....                      | 11        |
| 3.3. Implementacja klasyfikatorów.....                    | 12        |
| 3.3.1. SVC z jądrem RBF .....                             | 12        |
| 3.3.2. Las Losowy .....                                   | 12        |
| 3.3.3. Konwolucyjna sieć neuronowa .....                  | 13        |
| 3.4. Serializacja modeli .....                            | 13        |
| 3.5. Główne zależności.....                               | 14        |
| 3.6. Architektura logiczna .....                          | 14        |
| 3.7. Przetwarzanie danych .....                           | 14        |
| 3.8. Uczenie modeli .....                                 | 14        |
| 3.9. Ewaluacja jakości .....                              | 15        |
| 3.10. Inferencja: CSV i kamera .....                      | 15        |
| 3.11. Rola MediaPipe w systemie .....                     | 15        |
| <b>4. Eksperymenty.....</b>                               | <b>16</b> |
| 4.1. Cel eksperymentów .....                              | 16        |
| 4.2. Podział danych.....                                  | 16        |
| 4.3. Procedura trenowania i ewaluacji .....               | 16        |
| 4.3.1. Trenowanie SVC .....                               | 16        |
| 4.3.2. Trenowanie Lasu Losowego .....                     | 16        |
| 4.3.3. Trenowanie CNN .....                               | 17        |
| 4.3.4. Ewaluacja modeli .....                             | 17        |
| 4.4. Test w warunkach rzeczywistych .....                 | 17        |
| 4.4.1. Obserwacje jakościowe z inferencji kamerowej ..... | 17        |

|                                              |    |
|----------------------------------------------|----|
| <b>5. Wyniki i analiza</b>                   | 18 |
| 5.1. Dokładność na zbiorze testowym          | 18 |
| 5.2. Dyskusja wyników                        | 18 |
| 5.2.1. Przewaga CNN nad metodami klasycznymi | 18 |
| 5.2.2. Równoważność SVC i Lasu Losowego      | 18 |
| 5.3. Ograniczenia obecnej wersji             | 18 |
| <b>6. Wnioski</b>                            | 20 |
| 6.1. Podsumowanie realizacji                 | 20 |
| 6.2. Kierunki dalszego rozwoju               | 20 |
| <b>7. Instrukcja uruchomienia</b>            | 21 |
| 7.1. Kroki uruchomienia programu             | 21 |
| <b>Bibliografia</b>                          | 24 |
| <b>Spis rysunków</b>                         | 25 |
| <b>Spis tabel</b>                            | 26 |
| <b>Spis listingów</b>                        | 27 |

# 1. Cel projektu i wprowadzenie teoretyczne

## 1.1. Cel projektu

Celem ćwiczenia jest zaprojektowanie i zaimplementowanie systemu klasyfikacji statycznych gestów dłoni odpowiadających literom amerykańskiego języka migowego (ASL — *American Sign Language*). System o nazwie **mnist-translator** pracuje w czasie rzeczywistym, korzystając z obrazu z kamery internetowej, i jest w stanie rozpoznać 24 litery alfabetu ASL (A–Y, z wyłączeniem liter J i Z, które wymagają ruchu dłoni).

Szczegółowe cele projektu obejmują:

- zapoznanie się ze zbiorem danych Sign Language MNIST — jego strukturą, liczebnością klas i formatem próbek,
- opanowanie potoku przetwarzania obrazu: od przechwycenia klatki wideo przez detekcję dłoni za pomocą biblioteki MediaPipe, aż po ekstrakcję wektora cech dla klasyfikatora,
- wytrenowanie i porównanie trzech modeli uczenia maszynowego: **SVC z jądrem RBF**, **Lasu Losowego** oraz **konwolucyjnej sieci neuronowej (CNN)**,
- opracowanie modułowej aplikacji konsolowej umożliwiającej trenowanie, ewaluację i uruchamianie wnioskowania zarówno offline (z pliku CSV), jak i na żywo (z kamery),
- sformułowanie wniosków o możliwościach zastosowania klasyfikatorów pikselowych do rozpoznawania gestów w warunkach rzeczywistych.

## 1.2. Wprowadzenie teoretyczne

### 1.2.1. Zbiór danych Sign Language MNIST

Sign Language MNIST jest adaptacją oryginalnego zbioru MNIST cyfr [5] do zadania rozpoznawania liter języka migowego. Każda próbka to obraz dłoni o rozmiarze  $28 \times 28$  pikseli w skali szarości, spłaszczony do wektora 784 cech liczbowych (wartości pikseli 0–255).

**Tabela 1.1.** Statystyki zbioru danych Sign Language MNIST

| Podzbiór   | Liczba próbek | Liczba klas |
|------------|---------------|-------------|
| Treningowy | 27 455        | 24          |
| Testowy    | 7 172         | 24          |

Litery J (indeks 9) i Z (indeks 25) zostały wyłączone ze zbioru, ponieważ ich znaki ASL wymagają ruchu i nie mogą być reprezentowane przez pojedynczy obraz statyczny. Mapowanie etykiet na litery opisuje wzór:

$$\text{litera} = \text{chr}(\text{ord}('A') + \text{label}), \quad \text{label} \in \{0, 1, \dots, 8, 10, \dots, 24\}.$$

## 1.2.2. Metody klasyfikacji

### 1.2.2.1. Maszyna wektorów nośnych (SVM/SVC)

Maszyna wektorów nośnych [2] szuka hiperpłaszczyzny maksymalnie rozdzielającej klasy w przestrzeni cech. W przypadku nieliniowych granic decyzji stosowane są jądra (*kernels*), które implicitnie mapują dane do przestrzeni wyższego wymiaru. Jądro RBF (*Radial Basis Function*) definiowane jest jako:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2), \quad (1.1)$$

gdzie  $\gamma > 0$  jest parametrem szerokości jądra. Dla danych w przestrzeni 784-wymiarowej SVC z jądrem RBF jest w stanie uchwycić złożone relacje między pikselami bez jawnej inżynierii cech.

### 1.2.2.2. Las Losowy (*Random Forest*)

Las Losowy [1] jest metodą łączącą (*ensemble*) drzewa decyzyjne. Każde drzewo trenowane jest na losowej próbie danych (*bootstrap*) i korzysta z losowego podzbioru cech przy każdym podziale. Klasyfikacja odbywa się przez głosowanie większościowe:

$$\hat{y} = \arg \max_c \sum_{t=1}^T \mathbf{1}[h_t(\mathbf{x}) = c], \quad (1.2)$$

gdzie  $T$  to liczba drzew, a  $h_t$  to  $t$ -te drzewo decyzyjne. Las Losowy jest odporny na przeuczenie i nie wymaga normalizacji danych wejściowych.

### 1.2.2.3. Konwolucyjna sieć neuronowa (CNN)

Konwolucyjne sieci neuronowe [4] są szczególnie efektywne dla danych obrazowych, ponieważ uczą przestrzennych filtrów bezpośrednio z pikseli. Architektura zastosowana w projekcie składa się z dwóch bloków konwolucyjnych ( $\text{Conv2D} \rightarrow \text{MaxPool2D}$ ), warstwy spłaszczającej ( $\text{Flatten}$ ) i gęstej warstwy wyjściowej z funkcją *softmax*.

CNN operuje na siatce pikseli  $28 \times 28$ , co pozwala jej wychwytywać lokalne wzorce (krawędzie, narożniki, faktury) niedostępne dla klasyfikatorów operujących na spłaszczonym wektorze cech.

## 1.2.3. Detekcja dłoni — MediaPipe

MediaPipe [6] jest platformą Google do uczenia maszynowego na urządzeniach końcowych. Moduł *Gesture Recognizer* wykrywa dłoń w klatce wideo i zwraca 21 punktów kluczowych (*landmarks*) w przestrzeni 3D. Na ich podstawie wyznaczany jest prostokąt otaczający (*bounding box*), który jest przycinany i przeskalowywany do formatu  $28 \times 28$  pikseli.

## 1.2.4. Normalizacja cech

Wartości pikseli z zakresu  $[0, 255]$  są normalizowane do przedziału  $[0, 1]$  przez proste dzielenie:

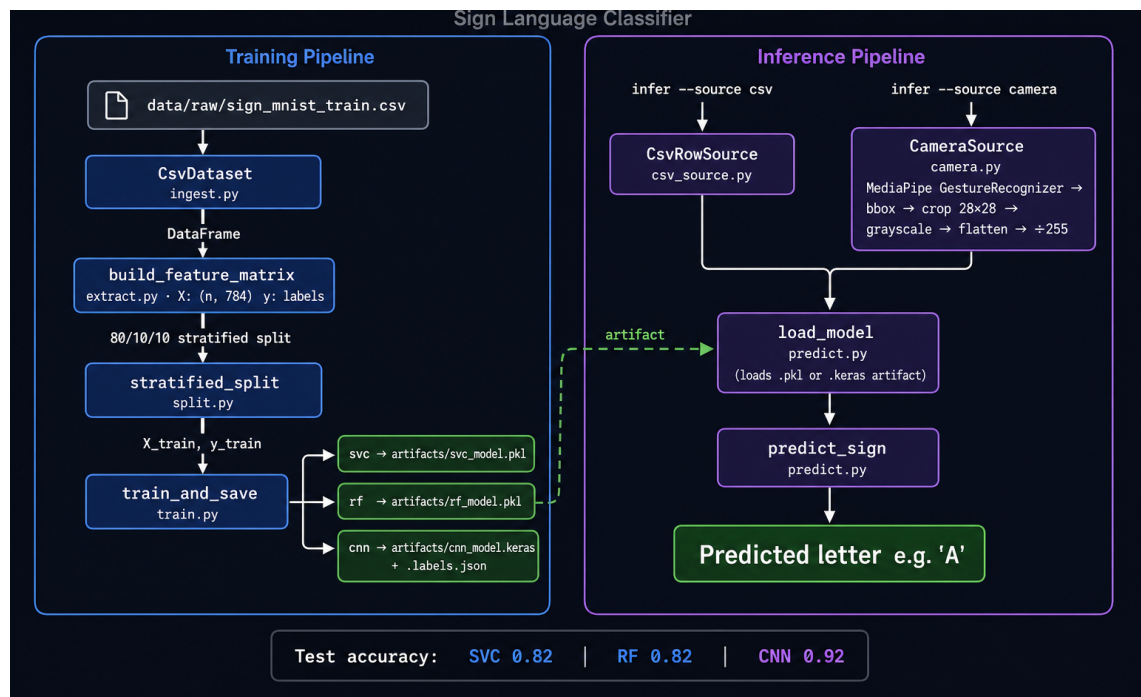
$$x' = \frac{x}{255,0}.$$

Zapewnia to zgodność formatu z danymi treningowymi ze zbioru Sign Language MNIST.

## 2. Opis zastosowanej metody

### 2.1. Ogólny schemat systemu

System **mnist-translator** działa w dwóch trybach: trenowania offline i wnioskowania (offline z CSV lub na żywo z kamery). Oba tryby korzystają z identycznego potoku ekstrakcji cech, co gwarantuje spójność przestrzeni cech. Schemat architektury systemu wykorzystany w projekcie znajduje się w katalogu `diagrams` i został przedstawiony na rysunku 2.1.



Rys. 2.1. Schemat systemu mnist-translator

### 2.2. Potok przetwarzania danych

Dane przepływają przez następujące moduły:

1. **Ingestion** (`src/data/ingest.py`) — wczytanie pliku CSV do struktury `DataFrame` i normalizacja pikseli do  $[0, 1]$ .
2. **Ekstrakcja cech** (`src/data/extract.py`) — budowa macierzy cech **X** (784 kolumny) i wektora etykiet **y**.
3. **Podział danych** (`src/data/split.py`) — stratyfikowany podział 80/10/10 na zbiory treningowy, walidacyjny i testowy.
4. **Trenowanie** (`src/model/train.py`) — dopasowanie wybranego klasyfikatora i zapis artefaktu do katalogu `artifacts/`.

5. **Ewaluacja** (`src/model/evaluate.py`) — wyznaczenie dokładności globalnej i raportu per-klasa.
6. **Inferencja** (`src/inference/`) — wczytanie modelu i klasyfikacja nowego wektora cech.

## 2.3. Warunki pomiarowe przy inferowaniu z kamery

Jakość predykcji w trybie kamerowym jest wrażliwa na warunki otoczenia. Tabela 2.1 zestawia zalecane ustawienia.

**Tabela 2.1.** Zalecane warunki podczas inferencji z kamery

| Parametr    | Zalecana wartość                              |
|-------------|-----------------------------------------------|
| Oświetlenie | Równomierne, bez twardych cieni na dłoni      |
| Dłoń        | Lewa, skierowana przodem do kamery            |
| Kamera      | DroidCam lub dowolna obsługiwana przez OpenCV |
| Tryb lustra | <b>Wyłączony</b> w aplikacji kamery           |
| Tło         | Jednolity, kontrastowy kolor                  |



## 3. Implementacja systemu

### 3.1. Struktura projektu

Listing 3.1. Struktura katalogów projektu mnist-translator

```
mnist-translator/
|-- main.py           # Punkt wejścia CLI (brak logiki biznesowej)
|-- pyproject.toml    # Zarządzanie zależnościami (uv)
|-- src/
|   |-- data/
|   |   |-- ingest.py    # Ładowanie CSV i normalizacja pikseli
|   |   |-- extract.py   # Budowa macierzy cech X i etykiet y
|   |   |-- split.py     # Stratyfikowany podział 80/10/10
|   |   |-- protocol.py  # Protokół DataSource
|   |-- model/
|   |   |-- train.py     # Trenowanie + zapis artefaktu
|   |   |-- evaluate.py  # Obliczanie metryk dokładności
|   |   |-- classifiers/
|   |       |-- svc.py    # Klasyfikator SVC z jądrem RBF
|   |       |-- random_forest.py # Las Losowy (200 drzew)
|   |       |-- cnn.py    # Konwolucyjna sieć neuronowa
|   |-- inference/
|       |-- camera.py    # Przechwytywanie wideo + MediaPipe
|       |-- csv_source.py # Źródło cech z wiersza CSV
|       |-- predict.py   # Ładowanie modelu i predykcja
|       |-- protocol.py  # Protokół FeatureSource
|-- tests/              # Testy jednostkowe (pytest)
|-- diagrams/          # Diagram architektury
|-- artifacts/         # Wytrenowane modele (gitignored)
```

### 3.2. Interfejs wiersza poleceń

Plik `main.py` udostępnia trzy tryby działania:

Tabela 3.1. Tryby działania aplikacji mnist-translator

| Tryb  | Opis                                     | Kluczowe opcje                                                                            |
|-------|------------------------------------------|-------------------------------------------------------------------------------------------|
| train | Trenuje klasyfikator i zapisuje artefakt | <code>--classifier {svc,rf,cnn}</code> , <code>--model</code> ,<br><code>--dataset</code> |
| eval  | Ewaluuje model na zbiorze testowym       | <code>--model</code> , <code>--dataset</code>                                             |
| infer | Inferencja z CSV lub z kamery            | <code>--source {csv,camera}</code> , <code>--row</code> ,<br><code>--camera</code>        |

Przykładowe użycie:

**Listing 3.2.** Przykłady uruchomien CLI

```
1 uv run python main.py train --classifier svc --model artifacts/svc_model.pkl
2 uv run python main.py eval --model artifacts/svc_model.pkl --dataset data/raw/
  sign_mnist_test.csv
3 uv run python main.py infer --source csv --row 42 --dataset data/raw/sign_mnist_test.csv
4 uv run python main.py infer --source camera --camera 0 --model artifacts/svc_model.pkl
```

## 3.3. Implementacja klasyfikatorów

### 3.3.1. SVC z jądrem RBF

**Listing 3.3.** Budowa potoku SVC (src/model/classifiers/svc.py)

```
1 from sklearn.svm import SVC
2 from sklearn.pipeline import Pipeline
3 from sklearn.preprocessing import StandardScaler
4
5 def build_svc() -> Pipeline:
6     """Buduje potok: standaryzacja + SVC z jądrem RBF."""
7     return Pipeline([
8         ("scaler", StandardScaler()),
9         ("svc", SVC(kernel="rbf", C=10, gamma="scale",
10             decision_function_shape="ovr")),
11     ])
```

### 3.3.2. Las Losowy

**Listing 3.4.** Budowa Lasu Losowego (src/model/classifiers/random\_forest.py)

```
1 from sklearn.ensemble import RandomForestClassifier
2
3 def build_random_forest() -> RandomForestClassifier:
4     """200 drzew, tryb OOB włączony."""
5     return RandomForestClassifier(
6         n_estimators=200,
7         random_state=42,
8         oob_score=True,
9         n_jobs=-1,
10    )
```

### 3.3.3. Konwolucyjna sieć neuronowa

**Listing 3.5.** Architektura CNN (src/model/classifiers/cnn.py)

```

1 import tensorflow as tf
2
3 def build_cnn(num_classes: int = 24) -> tf.keras.Model:
4     """Dwa bloki Conv2D->MaxPool + warstwa wyjściowa z softmax."""
5     model = tf.keras.Sequential([
6         tf.keras.layers.Input(shape=(28, 28, 1)),
7         tf.keras.layers.Conv2D(32, (3, 3), activation="relu",
8                                 padding="same"),
9         tf.keras.layers.MaxPooling2D(),
10        tf.keras.layers.Conv2D(64, (3, 3), activation="relu",
11                                padding="same"),
12        tf.keras.layers.MaxPooling2D(),
13        tf.keras.layers.Flatten(),
14        tf.keras.layers.Dense(128, activation="relu"),
15        tf.keras.layers.Dropout(0.3),
16        tf.keras.layers.Dense(num_classes, activation="softmax"),
17    ])
18    model.compile(optimizer="adam",
19                  loss="sparse_categorical_crossentropy",
20                  metrics=["accuracy"])
21    return model

```

## 3.4. Serializacja modeli

Modele scikit-learn są zapisywane i wczytywane za pomocą biblioteki `joblib`:

**Listing 3.6.** Zapis i odczyt modelu scikit-learn

```

1 import joblib
2
3 # Zapis modelu
4 joblib.dump(model, "artifacts/svc_model.pkl")
5
6 # Wczytanie modelu
7 model = joblib.load("artifacts/svc_model.pkl")

```

Model CNN jest zapisywany w formacie natywnym Keras (rozszerzenie `.keras`):

**Listing 3.7.** Zapis i odczyt modelu CNN (Keras)

```

1 # Zapis
2 model.save("artifacts/cnn_model.keras")
3
4 # Wczytanie
5 import tensorflow as tf
6 model = tf.keras.models.load_model("artifacts/cnn_model.keras")

```

### 3.5. Główne zależności

Tabela 3.2. Główne zależności projektu (pyproject.toml)

| Biblioteka           | Zastosowanie                        |
|----------------------|-------------------------------------|
| numpy, pandas        | Manipulacja danymi i macierzami     |
| scikit-learn         | SVC, Random Forest, metryki         |
| joblib               | Serializacja modeli                 |
| opencv-python        | Przechwytywanie klatek z kamery     |
| mediapipe            | Detekcja dłoni i punktów kluczowych |
| tensorflow >= 2.21.0 | Budowa i trenowanie CNN             |

### 3.6. Architektura logiczna

Projekt został podzielony na trzy warstwy:

- `src/data` – przygotowanie danych i podziały zbioru,
- `src/model` – uczenie i ewaluacja klasyfikatorów,
- `src/inference` – predykcja offline i online (kamera).

Warstwa wejściowa CLI znajduje się w `main.py` i deleguje logikę do modułów w `src/`. Takie podejście upraszcza testowanie i ogranicza sprzężenie między interfejsem użytkownika a logiką przetwarzania.

### 3.7. Przetwarzanie danych

Klasa `CsvDataset` ładuje plik CSV i wymusza spójność schematu (obecność kolumny `label` oraz dokładnie 784 cechy obrazu). Dla każdej próbki zwracana jest para:

```
(feature_vector: np.ndarray, label: str)
```

Wektor cech jest typu `float32` i przyjmuje wartości w  $[0, 1]$ . Etykiety numeryczne przekształcane są do liter poprzez mapowanie:

$$y_{\text{letter}} = \text{chr}(\text{ord}('A') + y_{\text{int}}) \quad (3.1)$$

Budowa macierzy cech realizowana jest przez `build_feature_matrix`, a podział danych przez `stratified_split`. Domyślnie stosowany jest podział 80/10/10 z zachowaniem proporcji klas.

### 3.8. Uczenie modeli

Moduł `src/model/train.py` obsługuje trzy warianty klasyfikatorów:

- **SVC (RBF)** – domyślny klasyfikator oparty o jądro radialne,
- **Random Forest** – las losowy o 200 drzewach,
- **CNN (Keras)** – sieć konwolucyjna dla wejścia  $28 \times 28$ .

Modele klasyczne serializowane są przez `joblib` do formatu `.pkl`. Model konwolucyjny zapisywany jest jako `.keras` oraz osobny plik mapowania etykiet `.labels.json`.

### 3.9. Ewaluacja jakości

Funkcja `evaluate` oblicza dokładność globalną oraz raport klasyfikacji `classification_report` biblioteki `scikit-learn` [8]. Wyniki zwracane są również jako słownik, co upraszcza dalsze automatyczne raportowanie.

### 3.10. Inferencja: CSV i kamera

Inferencja offline z jednego wiersza CSV realizowana jest przez `CsvRowSource`. Inferencja online wykorzystuje klasę `CameraSource`, która:

1. pobiera klatkę z kamery (OpenCV [7]),
2. wykrywa dłonie przez `MediaPipe GestureRecognizer` [3],
3. wyznacza obszar dłoni i dokonuje kadrowania,
4. przeskalowuje obraz do  $28 \times 28$  oraz normalizuje piksele,
5. przekazuje wektor 784 cech do modelu.

W przypadku braku detekcji dłoni moduł zwraca wektor zerowy, dzięki czemu interfejs inferencji pozostaje stabilny.

### 3.11. Rola MediaPipe w systemie

Kluczową rolą `MediaPipe` jest dostarczenie bounding boxa dłoni, dzięki czemu przycięty i przeskalowany fragment klatki jest znacznie bardziej zbliżony do próbek treningowych niż surowy obraz z kamery. Bez tego kroku predykcje byłyby praktycznie bezużyteczne. Jednocześnie `MediaPipe` bywa zawodny przy słabym oświetleniu lub gdy dłoń nie jest wyraźnie wyeksponowana.

## 4. Eksperymenty

### 4.1. Cel eksperymentów

Eksperymenty miały na celu porównanie trzech metod klasyfikacji pod względem dokładności na tych samych danych testowych oraz jakości predykcji w warunkach rzeczywistych (inferencja z kamery).

### 4.2. Podział danych

Dane treningowe z pliku `sign_mnist_train.csv` zostały podzielone stratyfikowanie na podzbiory:

- **Treningowy (80%)** — używany do dopasowania parametrów modelu.
- **Walidacyjny (10%)** — opcjonalnie do strojenia hiperparametrów.
- **Testowy (10%)** — do pomiaru końcowej dokładności.

Plik `sign_mnist_test.csv` (7 172 próbki) pełnił rolę niezależnego zbioru testowego wymaganego przez dokumentację projektu.

### 4.3. Procedura trenowania i ewaluacji

Dla każdego klasyfikatora procedura była identyczna:

1. Wczytanie `sign_mnist_train.csv` i normalizacja pikseli.
2. Stratyfikowany podział 80/10/10.
3. Trenowanie modelu na zbiorze treningowym.
4. Zapis artefaktu do katalogu `artifacts/`.
5. Ewaluacja na niezależnym zbiorze testowym.

#### 4.3.1. Trenowanie SVC

**Listing 4.1.** Polecenie trenowania klasyfikatora SVC

```
1 uv run python main.py train \  
2     --classifier svc \  
3     --model artifacts/svc_model.pkl \  
4     --dataset data/raw/sign_mnist_train.csv
```

#### 4.3.2. Trenowanie Lasu Losowego

**Listing 4.2.** Polecenie trenowania Lasu Losowego

```
1 uv run python main.py train \  
2     --classifier rf \  
3     --model artifacts/rf_model.pkl \  
4     --dataset data/raw/sign_mnist_train.csv
```

### 4.3.3. Trenowanie CNN

**Listing 4.3.** Polecenie trenowania CNN

```
1 uv run python main.py train \  
2   --classifier cnn \  
3   --model artifacts/cnn_model.keras \  
4   --dataset data/raw/sign_mnist_train.csv
```

### 4.3.4. Ewaluacja modeli

**Listing 4.4.** Ewaluacja modelu SVC na zbiorze testowym

```
1 uv run python main.py eval \  
2   --model artifacts/svc_model.pkl \  
3   --dataset data/raw/sign_mnist_test.csv
```

Wynikiem jest dokładność globalna oraz raport `classification_report` ze scikit-learn zawierający precyzję, czułość i miarę F1 dla każdej klasy.

## 4.4. Test w warunkach rzeczywistych

Po wytrenowaniu modeli przeprowadzono testy jakościowe z użyciem kamery internetowej, zachowując warunki zestawione w tabeli 2.1. Każdą z 24 liter testowano przez minimum 5 sekund, obserwując stabilność predykcji w oknie OpenCV.

**Listing 4.5.** Uruchomienie inferencji z kamery

```
1 uv run python main.py infer \  
2   --source camera \  
3   --camera 0 \  
4   --model artifacts/svc_model.pkl
```

#### 4.4.1. Obserwacje jakościowe z inferencji kamerowej

- **SVC** — predykcja stabilna w dobrym oświetleniu, wrażliwa na obrót dłoni i niejednolite tło.
- **Las Losowy** — podobna stabilność jak SVC; tendencja do nadmiernej pewności siebie przy niskiej jakości obrazu.
- **CNN** — najbardziej odporna predykcja; lepiej radzi sobie z lekkimi zmianami pozycji dłoni i oświetlenia.

We wszystkich modelach litery wizualnie podobne (np. M/N, R/U) powodowały częstsze błędy klasyfikacji.

## 5. Wyniki i analiza

### 5.1. Dokładność na zbiorze testowym

**Tabela 5.1.** Porównanie dokładności klasyfikatorów na zbiorze testowym Sign Language MNIST

| Model                  | Plik artefaktu            | Dok. testowa |
|------------------------|---------------------------|--------------|
| SVC (jądro RBF)        | artifacts/svc_model.pkl   | 0,82         |
| Las Losowy (200 drzew) | artifacts/rf_model.pkl    | 0,82         |
| CNN (Keras/TF)         | artifacts/cnn_model.keras | <b>0,92</b>  |

CNN osiąga wyraźnie wyższą dokładność (92%) w porównaniu z SVC i RF (oba 82%). Wynik jest spójny z literaturą — architektury konwolucyjne są naturalnie przystosowane do danych obrazowych.

### 5.2. Dyskusja wyników

#### 5.2.1. Przewaga CNN nad metodami klasycznymi

Różnica 10 punktów procentowych na korzyść CNN wynika z fundamentalnej cechy architektury: warstwy konwolucyjne uczą się hierarchicznych reprezentacji przestrzennych — krawędzi, narożników i wzorców wyższego rzędu — których płaski wektor 784 cech nie przekazuje. SVC i RF operują na przestrzeni 784-wymiarowej bez żadnego uwzględnienia sąsiedztwa pikseli.

#### 5.2.2. Równoważność SVC i Lasu Losowego

Zbliżona dokładność obu metod (82%) sugeruje, że granica decyzyjna jest podobnie złożona: wystarczająco skomplikowana, by wymagać nieliniowych metod, ale niewystarczająco złożona, by jedna metoda miała wyraźną przewagę bez dalszego strojenia hiperparametrów.

### 5.3. Ograniczenia obecnej wersji

Wersja aktualna jest rozwiązaniem klasyfikacji pojedynczych klatek i nie modeluje dynamiki ruchu. Oznacza to, że:

- **Znak J i Z** — znaki wymagające ruchu (np. J, Z) nie są klasyfikowane,
- **Brak mechanizmu wygładzania predykcji po czasie** — brak jest mechanizmu wygładzania predykcji po czasie,
- **Brak detekcji kontekstu sekwencji gestów** — brak detekcji kontekstu sekwencji gestów.
- **Wrażliwość na oświetlenie** — zmiana natężenia lub kierunku światła modyfikuje wartości pikseli i może prowadzić do błędnych predykcji.
- **Wrażliwość na pozycję** — dłoń musi być wycentrowana w kadrze i podobnej skali co próbki treninowe.



- **Brak niezmienności geometrycznej** — modele nie są z natury niezmiennicze na obrót, skalę ani przesunięcie; CNN częściowo kompensuje to przez operację *max pooling*.

Mimo tych ograniczeń projekt spełnia cel dydaktyczny i demonstracyjny: spójny pipeline od danych tablicowych do inferencji online.

## 6. Wnioski

### 6.1. Podsumowanie realizacji

W ramach projektu zrealizowano kompletny pipeline klasyfikacji znaków dłoni: od przygotowania danych tablicowych, przez uczenie i ewaluację modeli, po inferencję z kamery w czasie rzeczywistym. Implementacja zachowuje spójny format wejścia ( $28 \times 28$ , 784 cechy), co pozwala stosować różne klasyfikatory bez zmian interfejsu danych.

1. **CNN przewyższa metody klasyczne** na zadaniu klasyfikacji znaków ASL: osiąga 92% dokładności wobec 82% dla SVC i Lasu Losowego. Architektura konwolucyjna jest naturalnie dostosowana do danych obrazowych.
2. **SVC i Las Losowy osiągają identyczną dokładność (82%)** pomimo różnych strategii uczenia, co sugeruje, że obie metody osiągnęły podobną granicę dla tej reprezentacji danych.
3. **Jakość inferencji kamerowej zależy od warunków zewnętrznych** — oświetlenia, tła i pozycji dłoni — bardziej niż od wyboru klasyfikatora.
4. **MediaPipe odgrywa kluczową rolę** w przygotowaniu wejścia modelu; bez etapu detekcji dłoni system nie nadaje się do praktycznego zastosowania.
5. **Podejście pikselowe ma istotne ograniczenia praktyczne**. Reprezentacja oparta na punktach kluczowych mogłaby znacząco poprawić odporność systemu na zmienne warunki otoczenia.
6. **Modularna architektura projektu** (rozdzielenie warstw danych, modeli i inferencji) ułatwia wymianę komponentów i jest dobrą praktyką inżynierską w projektach uczenia maszynowego.

### 6.2. Kierunki dalszego rozwoju

Rozszerzenia, które mogą poprawić jakość i użyteczność systemu:

- dodanie modelu sekwencyjnego (np. LSTM/Transformer) dla rozpoznawania gestów dynamicznych,
- wygładzanie predykcji w oknie czasowym,
- augmentacja danych i rozszerzenie zbioru treningowego o rzeczywiste próbki z kamery,
- budowa modułu oceny wydajności czasu inferencji na różnych urządzeniach.

## 7. Instrukcja uruchomienia

### 7.1. Kroki uruchomienia programu

Poniższy przewodnik umożliwia samodzielne uruchomienie programu od konfiguracji środowiska po uruchomienie inferencji na żywo.

#### Krok 1: Pobranie repozytorium

**Listing 7.1.** Klonowanie repozytorium

```
git clone https://github.com/JustFiesta/mnist-translator.git
cd mnist-translator
```

#### Krok 2: Instalacja środowiska Python

**Listing 7.2.** Konfiguracja środowiska wirtualnego

```
# Instalacja Pythona 3.13 przez menedżera uv
uv python install 3.13
uv venv --python 3.13

# Windows PowerShell
.venv\Scripts\Activate.ps1

# Linux / macOS
source .venv/bin/activate

# Instalacja zależności z pliku lock
uv sync
```

#### Krok 3: Pobranie zbioru danych

1. Przejdź na stronę Kaggle:  
<https://www.kaggle.com/datasets/datamunge/sign-language-mnist/data>
2. Pobierz pliki `sign_mnist_train.csv` i `sign_mnist_test.csv`.
3. Utwórz katalog docelowy i umieść w nim pobrane pliki:

**Listing 7.3.** Przygotowanie struktury katalogów na dane

```
# Windows PowerShell
New-Item -ItemType Directory -Path data/raw -Force

# Linux / macOS
mkdir -p data/raw

# Skopiuj pobrane pliki CSV do katalogu data/raw/
```

## Krok 4: Trenowanie modeli

**Listing 7.4.** Trenowanie wszystkich trzech klasyfikatorów

```
# SVC (domyslny, najszybszy)
uv run python main.py train

# Las Losowy
uv run python main.py train --classifier rf --model artifacts/rf_model.pkl

# CNN (trenowanie moze zajac kilka minut)
uv run python main.py train --classifier cnn --model artifacts/cnn_model.keras
```

## Krok 5: Ewaluacja modeli

**Listing 7.5.** Ewaluacja wytrenowanych modeli

```
# Ewaluacja SVC
uv run python main.py eval \
    --model artifacts/svc_model.pkl \
    --dataset data/raw/sign_mnist_test.csv

# Ewaluacja Lasu Losowego
uv run python main.py eval \
    --model artifacts/rf_model.pkl \
    --dataset data/raw/sign_mnist_test.csv

# Ewaluacja CNN
uv run python main.py eval \
    --model artifacts/cnn_model.keras \
    --dataset data/raw/sign_mnist_test.csv
```

Zanotuj dokładność każdego modelu i porównaj z wynikami z tabeli 5.1.

## Krok 6: Inferencja offline (z pliku CSV)

**Listing 7.6.** Inferencja na wybranym wierszu pliku CSV

```
uv run python main.py infer \
    --source csv \
    --row 42 \
    --dataset data/raw/sign_mnist_test.csv \
    --model artifacts/svc_model.pkl
```

## Krok 7: Inferencja na żywo z kamery

**Listing 7.7.** Uruchomienie inferencji w czasie rzeczywistym

```
uv run python main.py infer \
  --source camera \
  --camera 0 \
  --model artifacts/svc_model.pkl
```

### Uwagi:

- Przy pierwszym uruchomieniu aplikacja automatycznie pobierze plik modelu MediaPipe (`gesture_recognizer.task`) — wymagany jest dostęp do Internetu.
- Naciśnij klawisz `q`, aby zakończyć działanie okna kamery.
- Jeśli kamera nie jest wykrywana pod indeksem 0, spróbuj opcji `--camera 1`.

## Krok 8: Uruchomienie testów jednostkowych

**Listing 7.8.** Uruchamianie testów pytest

```
# Wszystkie testy
uv run pytest

# Testy z raportem pokrycia kodu
uv run pytest --cov=src --cov-report=term-missing
```

# Bibliografia

- [1] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [2] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995.
- [3] Google MediaPipe. Mediapipe solutions and tasks, 2026. Dostęp: 2026-06-13.
- [4] Yann LeCun, Bernhard Boser, John S. Denker, Donnie Henderson, Richard E. Howard, Wayne Hubbard, and Lawrence D. Jackel. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1(4):541–551, 1989.
- [5] Yann LeCun, Leon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- [6] Camillo Lugaresi et al. Mediapipe: A framework for building perception pipelines. *arXiv preprint arXiv:1906.08172*, 2019.
- [7] OpenCV Team. Opencv documentation, 2026. Dostęp: 2026-06-13.
- [8] scikit-learn developers. scikit-learn documentation, 2026. Dostęp: 2026-06-13.

# Spis rysunków

|     |                                            |   |
|-----|--------------------------------------------|---|
| 2.1 | Schemat systemu mnist-translator . . . . . | 9 |
|-----|--------------------------------------------|---|

# Spis tabel

|     |                                                                                     |    |
|-----|-------------------------------------------------------------------------------------|----|
| 1.1 | Statystyki zbioru danych Sign Language MNIST . . . . .                              | 7  |
| 2.1 | Zalecane warunki podczas inferencji z kamery . . . . .                              | 10 |
| 3.1 | Tryby działania aplikacji mnist-translator . . . . .                                | 11 |
| 3.2 | Główne zależności projektu (pyproject.toml) . . . . .                               | 14 |
| 5.1 | Porównanie dokładności klasyfikatorów na zbiorze testowym Sign Language MNIST . . . | 18 |



## Spis listingów

|     |                                                                         |    |
|-----|-------------------------------------------------------------------------|----|
| 3.1 | Struktura katalogów projektu mnist-translator . . . . .                 | 11 |
| 3.2 | Przykłady uruchomien CLI . . . . .                                      | 12 |
| 3.3 | Budowa potoku SVC (src/model/classifiers/svc.py) . . . . .              | 12 |
| 3.4 | Budowa Lasu Losowego (src/model/classifiers/random_forest.py) . . . . . | 12 |
| 3.5 | Architektura CNN (src/model/classifiers/cnn.py) . . . . .               | 13 |
| 3.6 | Zapis i odczyt modelu scikit-learn . . . . .                            | 13 |
| 3.7 | Zapis i odczyt modelu CNN (Keras) . . . . .                             | 13 |
| 4.1 | Polecenie trenowania klasyfikatora SVC . . . . .                        | 16 |
| 4.2 | Polecenie trenowania Lasu Losowego . . . . .                            | 16 |
| 4.3 | Polecenie trenowania CNN . . . . .                                      | 17 |
| 4.4 | Ewaluacja modelu SVC na zbiorze testowym . . . . .                      | 17 |
| 4.5 | Uruchomienie inferencji z kamery . . . . .                              | 17 |
| 7.1 | Klonowanie repozytorium . . . . .                                       | 21 |
| 7.2 | Konfiguracja środowiska wirtualnego . . . . .                           | 21 |
| 7.3 | Przygotowanie struktury katalogów na dane . . . . .                     | 21 |
| 7.4 | Trenowanie wszystkich trzech klasyfikatorów . . . . .                   | 22 |
| 7.5 | Ewaluacja wytrenowanych modeli . . . . .                                | 22 |
| 7.6 | Inferencja na wybranym wierszu pliku CSV . . . . .                      | 22 |
| 7.7 | Uruchomienie inferencji w czasie rzeczywistym . . . . .                 | 23 |
| 7.8 | Uruchamianie testów pytest . . . . .                                    | 23 |